🚀 Kubernetes for Application Developers

Why We Use It

Intermediate Kubernetes — Module 1 of 13

🎯 Module Objectives

- Articulate the developer value proposition of Kubernetes
- Trace the journey from source code to running cluster workload
- Define the developer-platform contract and responsibilities
- Review core abstractions (Pod, Deployment, Service) through a developer lens
- Compare declarative vs. imperative approaches
- Connect 12-factor app principles to Kubernetes patterns
- Preview the full 13-module course roadmap

🤔 The Problem We're Solving

Why developers needed something better

💔 Before Kubernetes — Developer Pain Points

Deployment Challenges

- "Works on my machine" syndrome
- Manual server provisioning tickets
- Environment drift between dev/staging/prod
- Multi-hour deployment windows

Operational Burden

- Developers on-call for infrastructure issues
- Scaling requires capacity planning meetings
- Rollback = panic + SSH + hope
- No self-service for infrastructure needs

✅ After Kubernetes — Developer Gains

Deployment Gains

- Containers guarantee environment parity
- Self-service deployments via `kubectl apply`
- Same manifest works dev → staging → prod
- Rolling updates with zero downtime

Operational Gains

- Platform team owns infrastructure
- Auto-scaling based on actual metrics
- Rollback = `kubectl rollout undo`
- Declarative config = version-controlled infra

📊 Before vs. After — Side by Side

Concern	Before K8s	With K8s
Deploy a new version	SSH, scripts, prayer	`kubectl apply -f deployment.yaml`
Scale to handle load	File a ticket, wait days	HPA scales automatically
Rollback a bad release	Restore from backup, redeploy	`kubectl rollout undo`
Service discovery	Hardcoded IPs, load balancer configs	DNS-based: `<service>.<namespace>.svc.cluster.local`
Environment parity	Snowflake servers	Same container image everywhere
Health monitoring	External monitoring + manual restart	Liveness/readiness probes + auto-restart

🛠️ The Developer Workflow

From code to cluster

🚀 Code to Cluster — The Journey

The Developer Pipeline

1. **Write Code** — Application logic, tests, Dockerfile
2. **Build Image** — `docker build -t myapp:v1.2.3 .`
3. **Push to Registry** — `docker push ecr-repo/myapp:v1.2.3`
4. **Declare Desired State** — Write/update Kubernetes manifests
5. **Apply to Cluster** — `kubectl apply -f k8s/`
6. **Kubernetes Reconciles** — Controller loop makes it so
7. **Observe & Iterate** — Logs, metrics, traces

📝 What Developers Own

Application Artifacts

- Application source code and tests
- Dockerfile / container definition
- Application configuration

Kubernetes Manifests

- Deployment and Service definitions
- ConfigMaps and Secrets
- Resource requests, limits, and health probes

> **Key Insight:** Developers define *what* they need. The platform provides *how* it's delivered.

🤝 The Developer-Platform Contract

Clear boundaries, clear responsibilities

👤 Developer vs. Platform Responsibilities

Developer Responsibility	Platform Responsibility
Container image (Dockerfile)	Container runtime (containerd)
Resource requests & limits	Node capacity & scheduling
Health check endpoints	Health check execution & remediation
Deployment strategy choice	Rolling update orchestration
Service port definitions	Network routing & load balancing
PVC requests (size, access mode)	Storage provisioning (EBS, EFS)
Scaling thresholds (HPA config)	Cluster autoscaler, node pools

📜 The Kubernetes API — Your Contract Interface

The API Server is the single point of interaction

`kubectl`, CI/CD pipelines, controllers, and the scheduler all speak the same API.

```
# Every developer action is an API call
kubectl apply -f deployment.yaml    # POST/PUT to /apis/apps/v1/deployments
kubectl get pods                    # GET to /api/v1/pods
kubectl logs my-pod                 # GET to /api/v1/pods/my-pod/log
kubectl port-forward svc/myapp 8080 # Upgrade to SPDY/WebSocket
```

Why this matters: The API is versioned and consistent. Your manifests work on future clusters thanks to API compatibility guarantees.

🆕 Self-Service Infrastructure

Infrastructure requests become API calls -- no tickets required:

I need...

- A load balancer
- Persistent storage
- A cron job
- DNS entry and TLS termination
- Horizontal scaling

I write...

- `Service type: LoadBalancer`
- `PersistentVolumeClaim`
- `CronJob`
- `Service` + `Ingress` with TLS
- `HorizontalPodAutoscaler`

📦 Core Abstractions Review

Pods, Deployments, and Services — the developer lens

🗺️ Abstraction-to-Need Mapping

Developer Need	K8s Abstraction	What It Provides
Run my container	Pod	Smallest deployable unit, shared network/storage
Keep N replicas running	Deployment / ReplicaSet	Desired state, rolling updates, rollback
Stable network endpoint	Service	Load balancing, DNS, stable IP
External traffic routing	Ingress / Gateway	HTTP routing, TLS, path-based routing
Configuration management	ConfigMap / Secret	Externalized config, decoupled from image
Persistent data	PVC / StorageClass	Dynamic provisioning, lifecycle management
Run-to-completion tasks	Job / CronJob	Batch processing, scheduled tasks

📦 Pods — The Atomic Unit

- Smallest deployable unit in K8s
- One or more containers sharing network & storage
- Every container in a pod shares `localhost`
- Pods are **ephemeral** — they can be killed and rescheduled
- You rarely create pods directly — use Deployments

```yaml
kind: Pod
spec:
  containers:
  - name: myapp
    image: ecr-repo/myapp:v1.2.3
    ports: [{ containerPort: 8080 }]
    resources:
      requests: { cpu: "250m", memory: "256Mi" }
    # ... livenessProbe, readinessProbe
```

⚙️ Deployments — Desired State Management

```yaml
kind: Deployment
spec:
  replicas: 3
  strategy:
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    spec:
      containers:
      - image: ecr-repo/myapp:v1.2.3
        # ... ports, resources, probes
```

What Deployments provide:

- **Desired state** — declare replica count and Kubernetes reconciles continuously
- **Rolling updates** — new pods start before old ones terminate (`maxUnavailable: 0`)
- **Rollback** — instantly revert to a previous revision with `kubectl rollout undo`
- **Self-healing** — failed pods are automatically replaced to match the desired count

🔗 Services — Stable Networking

```yaml
kind: Service
spec:
  selector: { app: myapp }
  ports:
  - port: 80
    targetPort: 8080
  type: ClusterIP
```

Why Services exist:

- Pods have dynamic IPs — they change on restart
- Services provide a **stable** virtual IP
- Built-in load balancing across pod replicas
- DNS-based discovery — no hardcoded IPs
- Selector connects Service to Pods via labels

Think of it as: A stable address that always routes to healthy pods.

📜 Declarative vs. Imperative

The most important mental shift

🔄 Two Ways to Interact with Kubernetes

🚨 Imperative (Not Recommended)

```
# Step-by-step commands
kubectl create deployment myapp \
  --image=myapp:v1
kubectl scale deployment myapp \
  --replicas=3
kubectl set image deployment/myapp \
  myapp=myapp:v2
```

Tells Kubernetes *what to do* step by step.

✅ Declarative (Best Practice)

```
# Declare desired state
kubectl apply -f deployment.yaml

# Update? Change the YAML, re-apply
kubectl apply -f deployment.yaml

# Everything is in version control
git diff deployment.yaml
```

Tells Kubernetes *what you want* — it figures out how.

🏆 Why Declarative Wins

🔎 Auditability

- Git history and PR reviews for every change
- Blame shows who changed what

🔄 Reproducibility

- Same YAML = same result everywhere
- Disaster recovery is a `kubectl apply`

🤖 Automation

- GitOps workflows (ArgoCD, Flux)
- CI/CD pipelines with drift detection

🌐 Environment Parity

Same image, same behavior, everywhere

📋 Environment Parity with Kubernetes

The Promise

The same container image runs identically in every environment. Configuration is injected, not baked in.

What stays the same

- Container image (immutable artifact)
- Application code, dependencies, and runtime
- OS-level packages

What changes per environment

- ConfigMaps and Secrets
- Resource requests/limits and replica counts
- Ingress rules / hostnames

🛠️ Environment Parity in Practice

```
# Directory structure for multi-environment manifests
k8s/
├── base/
│   ├── deployment.yaml        # Common Deployment spec
│   ├── service.yaml           # Common Service spec
│   └── kustomization.yaml     # Base kustomization
├── overlays/
│   ├── dev/
│   │   ├── kustomization.yaml # Dev-specific patches
│   │   └── configmap.yaml
│   ├── staging/
│   │   └── ...                # Same structure as dev
│   └── prod/
│       └── ...                # Same structure as dev
```

```
# Deploy to each environment
kubectl apply -k k8s/overlays/dev/
kubectl apply -k k8s/overlays/staging/
kubectl apply -k k8s/overlays/prod/
```

🎯 12-Factor App Principles

How Kubernetes embodies modern application design

📚 12-Factor Principles & Kubernetes

Factor	Principle	Kubernetes Implementation
I	Codebase	One repo, one container image
II	Dependencies	Bundled in container image
III	Config	ConfigMaps, Secrets, env vars
IV	Backing Services	Services, ExternalName, endpoints
V	Build, Release, Run	Image build → tag → Deployment
VI	Processes	Stateless pods, external state

📚 12-Factor Principles & Kubernetes (cont.)

Factor	Principle	Kubernetes Implementation
VII	Port Binding	`containerPort` + Service
VIII	Concurrency	Horizontal scaling (replicas, HPA)
IX	Disposability	Fast startup, graceful shutdown (`preStop`)
X	Dev/Prod Parity	Same image + Kustomize overlays
XI	Logs	Write to stdout/stderr, collected by platform
XII	Admin Processes	Jobs, CronJobs, `kubectl exec`

Design implication: Applications that follow these principles are "Kubernetes-native" — they leverage the platform fully and avoid fighting it.

🚫 Factor IX Deep Dive — Disposability

```yaml
spec:
  terminationGracePeriodSeconds: 30
  containers:
  - name: myapp
    lifecycle:
      preStop:
        exec:
          command: ["/bin/sh", "-c", "sleep 5"]
    # 1. Removed from endpoints → preStop runs → SIGTERM
    # 2. Wait terminationGracePeriodSeconds → SIGKILL
```

Graceful shutdown sequence:

- Pod is removed from Service endpoints — no new traffic is routed
- **preStop hook** runs first — the `sleep 5` allows in-flight requests to drain
- **SIGTERM** is sent to the container process — the app should finish work and exit
- After `terminationGracePeriodSeconds`, Kubernetes sends **SIGKILL** to force termination

☁️ Kubernetes on EKS

Your enterprise platform

☁️ Amazon EKS — What It Means for Developers

AWS Manages

- Control plane (API server, etcd, scheduler, controllers)
- HA and multi-AZ availability
- Version upgrades and certificates

Your Team Manages

- Worker node groups (EC2 or Fargate)
- Application deployments and configuration
- Namespaces, RBAC, and add-ons

> **EKS advantage:** The control plane is someone else's problem. You focus on workloads.

🛠️ EKS Developer Workflow

```
# 1. Authenticate to the cluster
aws eks update-kubeconfig --name platform-lab --region us-east-2

# 2. Verify connectivity and set namespace
kubectl cluster-info
kubectl config set-context --current --namespace=my-team

# 3. Deploy your application
kubectl apply -f k8s/

# 4. Monitor your deployment
kubectl rollout status deployment/myapp
kubectl get pods -w

# 5. Check logs and debug
kubectl logs -l app=myapp --tail=100 -f
kubectl describe pod myapp-abc123
```

🏗️ Enterprise Architecture Patterns

Mapping N-tier applications to Kubernetes primitives

🏗️ Common N-Tier Application on Kubernetes

🌐 Frontend

- **Deployment** — stateless replicas
- **Service** — internal routing
- **Ingress** — external traffic + TLS

⚙️ API / Backend

- **Deployment** — horizontally scaled
- **Service** — DNS discovery
- **ConfigMap / Secret** — config

🗃️ Data Layer

- **StatefulSet + PVC** — stable storage
- **Headless Service** — pod DNS
- **Cache** — Redis / Memcached

Every tier maps to a small set of primitives -- master these and you can model any architecture.

Enterprise Architecture on EKS

- **DNS-critical services** — multi-AZ scheduling + PodDisruptionBudgets
- **Dual Ingress** — separate internal vs. external traffic
- **GitOps via FluxCD** — Git as single source of truth
- **Vault + ESO** — secrets management with Kubernetes auth
- **ECR** — scanned container images
- **Observability** — Prometheus, Grafana, Splunk

Each pattern is covered hands-on in Modules 5-13.

🗺️ Course Roadmap

#	Module	Focus
1	**Why Kubernetes**	Core abstractions, EKS
2	Scaling	HPA, VPA, Cluster Autoscaler
3	Storage	PVCs, StatefulSets
4	Services	Service types, DNS
5	Config & Secrets	ConfigMaps, Secrets, Vault
6	Ingress & Egress	Ingress, Gateway API, TLS
7	RBAC & Security	Roles, PSA, IRSA
8	Network Policies	Pod-to-pod traffic control

🗺️ Course Roadmap — Modules 9-13

#	Module	Focus
9	Observability	Logging (Splunk), metrics, Grafana, alerting
10	Health Probes & Debugging	Liveness, readiness, failure states, troubleshooting
11	Deployment Strategies	Rolling, blue-green, canary, progressive delivery
12	Helm & Kustomize	Charts, templating, release management
13	CI/CD & GitOps	Jenkins, ArgoCD, FluxCD

🔧 Lab 1: Exploring Your Kubernetes Cluster

Hands-on: Connect, explore, deploy

- Connect to the EKS cluster using `aws eks update-kubeconfig`
- Explore cluster resources with `kubectl get` and `kubectl describe`
- Deploy a sample application with `kubectl create deployment`
- Expose it with a Service and verify connectivity
- Scale the Deployment up and down and observe replica changes
- Examine pod lifecycle events and logs

> **Duration:** ~45 minutes

📚 Module 1 — Key Takeaways

- **Kubernetes is a developer platform** — it transforms infrastructure requests into self-service API calls
- **The developer-platform contract** — you define what you need; the platform delivers how
- **Core abstractions map to developer needs** — Pod (run), Deployment (manage), Service (connect)
- **Declarative > imperative** — YAML in Git is your source of truth
- **Environment parity via containers** — same image, different config per environment
- **12-factor apps thrive on K8s** — the platform was designed for this pattern
- **EKS simplifies operations** — AWS manages the control plane, you manage workloads
- **N-tier apps map cleanly to K8s primitives** — Deployment, Service

❓ Questions & Discussion

What aspects of your current workflow would you most like Kubernetes to improve?

Up Next: Module 2 — Scaling Your Applications